

The STL Map Classes

CS101 Introduction to Intermediate Programming CPU Spring 2026

Table of Contents

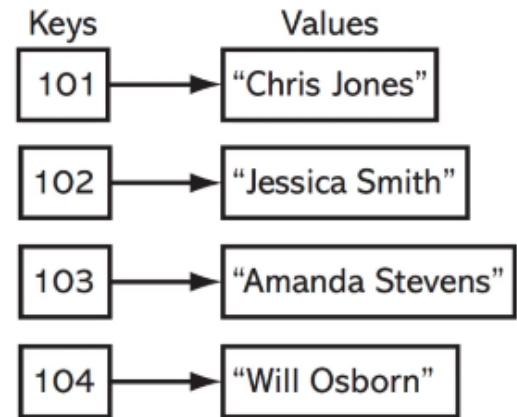
- [1. Objectives](#)
- [2. Overview](#)
- [3. Simple example](#)
- [4. How to get a map](#)
- [5. Member functions of `std::map`](#)
- [6. PRACTICE Initialize a map](#)
- [7. PRACTICE Initialize a map with an initialization list](#)
- [8. PRACTICE Adding elements to an existing map using operator `\[\]`](#)
- [9. Adding values to a map with `insert`](#)
- [10. Adding elements to a map with `emplace`](#)
- [11. Finding values in a map with `find` using a key](#)
- [12. Retrieving values from a map with `at` using a key](#)
- [13. Difference between retrieval with `find` or `at`](#)
- [14. Deleting map elements](#)
- [15. PRACTICE Using `emplace`, `find`, and `erase`](#)
- [16. Iterating over a map with a range-based loop](#)
- [17. PRACTICE Structured bindings](#)
- [18. Using an iterator with a map](#)
- [19. PRACTICE Using an iterator with a map](#)
- [20. Storing a vector as values in a map](#)
- [21. PRACTICE Student scores with structured binding](#)
- [22. PRACTICE Exercise: Inventory Prices](#)
- [23. Inventory Prices: Solutions](#)
- [24. Storing objects of a class as values in a map](#)
- [25. PRACTICE Exercise: Storing Class Objects in a Map](#)
- [26. Solution: Storing Book class objects in a map](#)
- [27. Storing objects of a class as a key to a map](#)
- [28. PRACTICE Storing class information as map key](#)
- [29. Other map classes: `unordered\_map`, `multimap`, `set`](#)
- [30. Review questions](#)
- [31. Glossary](#)

1. Objectives

The map data structure is a **balanced binary search tree**.

- [] Overview of the map class
- [] How to get a map
- [] Member functions of `std::map`
- [] Adding to, retrieving from and deleting map elements
- [] Using iterators with a map
- [] Storing class objects in a map as key and value
- [] Storing map objects in a vector
- [] Other map classes: `multimap`, `unordered_map`

2. Overview



#caption: Employee numbers (key) mapped to names (values).

- What is a map?

A map is an associative container whose elements are key-value pairs. To retrieve a value from a map, you use the key that is associated with that value - like looking up a word in a dictionary where the words are the keys, and the definitions are the values. A related concept is the parallel array.

- Why use map objects?

Maps allow fast key-based lookup. Elements are maintained in sorted key order and keys cannot be duplicated. Perfect for symbol tables (like variable-type-address, frequency counting (e.g. tracking how many times a word occurs in a text), and settings storage (e.g. "theme" -> "dark").

- How is a map **key** different from an **index**, which is also unique, cannot be duplicated, and automatically ordered?

Maps generalize the idea of an index: the key can be any type (string, class, etc.) not just integers, and storage does not need to be contiguous or pre-sized.

- What do you have to do to use map?

You have to #include the <map> header and use the std::map<Key, Value> syntax.

- Conceptual ideas:

- **Abstraction** gives you the tree: the idea or interface of a Product without its implementation details.
- **Encapsulation** protects the tree, controlling access to internal structure through public member functions.
- **Generalization** gives you many different trees, e.g. a map of Product objects, a vector of Product pointers etc.
- **Specialization** lets you refine the tree, e.g. PerishableProduct, or DigitalProduct by extending Product with special attributes.

3. Simple example

- This program generates the map shown in the figure earlier.

```

#include <iostream>
#include <map>
#include <string>
using namespace std;

int main()
{
    // Create a map of employee IDs and associated names.
    map<int, string> employees;
    employees[101] = "Jimmy Jones";
    employees[102] = "Albert Smith";
    employees[103] = "Jenny Brown";
    employees[104] = "Johnny Miller";

    // Create a map iterator.
    map<int, string>::iterator iter;

    // Use the iterator to display each map element.
    for (iter = employees.begin(); iter != employees.end(); iter++)
    {
        cout << "ID: " << iter->first << ", "
              << "Name: " << iter->second << endl;
    }
    return 0;
}

```

```

ID: 101, Name: Jimmy Jones
ID: 102, Name: Albert Smith
ID: 103, Name: Jenny Brown
ID: 104, Name: Johnny Miller

```

- How are the key-value pairs accessed?

The map key-value pairs can be accessed using the overloaded [] operator. A for loop using an iterator returns a pointer to each key-value pair. The key is accessed with first, and the value with second.

- You could access key and value also by explicit dereferencing:

```

iter->first == (*iter).first
iter->second == (*iter).second

```

4. How to get a map

You can get a map in three ways:

1. With a **default constructor**: as seen in the simple example. This creates an empty map.

```

map<keyDatatype, valueDatatype> name;

```

2. With a **range constructor**: Creates a map object that contains a range of values specified by two iterators (bookmarks).

```
map<keyDatatype, valueDatatype> name(iterator1, iterator2);
```

Example: Using an initializer list to create the map.

```
map<int, string> fruit {{1,"apple"},{2,"banana"},{3,"cherry"}};
map<int, string> fruit2(fruit.begin(), fruit.end()); // copy fruit
for (auto iter=fruit2.rbegin();iter!=fruit2.rend();iter++) // reverse iterator
{
    cout << iter->first << " ";
    cout << iter->second << endl;
}
```

3. With another map (copy constructor): the object is a map that is a copy of another map object. Key and value data types must be the same.

```
map<keyDatatype, valueDatatype> name(map2);
```

Example:

```
map<int,string> map2 {{1,"jack"},{2,"millie"},{3,"nanny"},{4,"poppy"},{5,"rosie"}}
map<int,string> map1(map2);
map<int,string>::iterator it;
for (auto iter=map1.begin();iter!=map1.end();iter++)
    cout << iter->first << ", " << iter->second << endl;
```

5. Member functions of `std::map`

Most of these are the usual suspects that you already know from the vector class because though associative, a map is still a **container**.

See cppreference.com for a complete list of member functions.

1. Capacity
 - empty()**
true if map is empty
 - size()**
number of key-value pairs
2. Element access
 - at(key)**
access value at key (throws if key missing)
 - operator[]**
access or insert value by key
3. Modifiers
 - insert(pair)**
insert key-value pair in-place
 - emplace(pair)**
construct & insert element in-place
 - erase(key)**
erase by key
 - clear()**

- remove all elements
- swap(other_map)**
exchange contents with another map
- 4. Lookup
 - find(key)**
returns iterator to key or end()
 - count(key)**
returns 1 if key exists, else 0
- 5. Iterators
 - begin(), end()**
forward iteration
 - rbegin(), rend()**
reverse iteration
- 6. Range
 - lower_bound(key)**
first key $\geq$ given key
 - upper_bound(key)**
first key $>$ given key
 - equal_range(key)**
pair of lower/upper bound iterators

6. PRACTICE Initialize a map

- **Task:** Create a map called `capitals` that stores the capital cities of three countries, for example Germany, France, Italy. Then, use a for loop with an iterator to print each country and its capital. Use the starter code below.
- **Sample output:**

```
Country: France. Capital: Paris
Country: Germany. Capital: Berlin
Country: Italy. Capital: Rome
```

- **Requirements:**
 1. Use the `[]` operator to insert elements into the map.
 2. Use an iterator-based for loop to access and print the entries.
- **Starter Code:** tinyurl.com/init-map

```
// Create a map called `capitals` that stores the capital cities of
// Germany, France, Italy as values. Initialize the map using []. Use
// a for loop with an iterator to print each country and its capital.

// header files

using namespace std;

int main()
{
    // Create a map for capitals of countries.

    // Initialize the map with the [] operator.

    // Create an iterator.
```

```

    // Loop over the map using the iterator to display key-value pairs.
    return 0;
}

```

- **Solution:**

```

// Create a map called `capitals` that stores the capital cities of
// Germany, France, Italy as values. Initialize the map using []. Use
// a for loop with an iterator header files to print each country and
// its capital.

// header files
#include <iostream>
#include <string>
#include <map>
using namespace std;

int main()
{
    // Create a map for capitals of countries.
    map<string,string> capitals;

    // Initialize the map with the [] operator.
    capitals["Germany"] = "Berlin";
    capitals["France"] = "Paris";
    capitals["Italy"] = "Rome";

    // Create an iterator.
    auto it = capitals.begin();

    // Loop over the map using the iterator to display key-value pairs.
    for (it=capitals.begin(); it!=capitals.end();it++)
    {
        cout << "Country: " << it->first
              << ". Capital: " << it->second << endl;
    }
    return 0;
}

```

```

Country: France. Capital: Paris
Country: Germany. Capital: Berlin
Country: Italy. Capital: Rome

```

- What is the type of auto for the iterator in the line in the code: `auto it=capitals.begin();` ?

The type of `auto it = capitals.begin();` is `map<string,string>::iterator`. The line is identical to: `map<string,string>::iterator it=capitals.begin();`

7. **PRACTICE** Initialize a map with an initialization list

- Change your previous program: add another map called `capitals2` with the capitals of the USA, China, Russia but initialize it with a list.

Example: The following map `emp` is initialized with a list whose entries are comma-separated.

```
map<int,string> emp =
    { {101, "Jimmy Jones"},
      {102, "Albert Smith"},
      {103, "Jenny Brown"},
      {104, "Johnny Miller"} };

for (auto it=emp.begin(); it!=emp.end();it++)
    cout << it->first << " - " << it->second << endl;
```

```
101 - Jimmy Jones
102 - Albert Smith
103 - Jenny Brown
104 - Johnny Miller
```

- Sample output:

```
Country: China. Capital: Bejing
Country: Russia. Capital: Moscow
Country: USA. Capital: Washington D.C.
```

- What happens when you duplicate a key?

If two or more elements in an initialization list have the same key, only the first will be added to the map, the others will be ignored.

- Solution:

```
// Create a map called `capitals2` that stores the capital cities of
// USA, China, and Russia as values. Initialize the map using an
// initializer list. Use a for loop with an iterator header files to
// print each country and its capital.

// header files
#include <iostream>
#include <string>
#include <map>
using namespace std;

int main()
{
    // Create a map for capitals of countries.
    map<string,string> capitals2 =
        { {"USA", "Washington D.C."},
          {"China", "Bejing"},
          {"Russia", "Moscow"} };

    // Create an iterator.
    auto it = capitals2.begin();

    // Loop over the map using the iterator to display key-value pairs.
    for (it=capitals2.begin(); it!=capitals2.end();it++)
    {
        cout << "Country: " << it->first
              << ". Capital: " << it->second << endl;
    }
    return 0;
}
```

Country: China. Capital: Beijing
Country: Russia. Capital: Moscow
Country: USA. Capital: Washington D.C.

8. PRACTICE Adding elements to an existing map using operator []

- The following program adds a couple of employees to an already existing map and displays the content of the map.
- For demonstration purposes, we use the full iterator definition (not the auto shortcut) and a while loop with an initialized iterator:
- Add meaningful comments where indicated (//...) to explain what exactly is going on.
- Starter code: tinyurl.com/operator-map

```
#include <iostream>
#include <map> // ...
using namespace std;
int main()
{
    // ...
    map<int,string> employees = {{101,"Jim Jones"},"{102,"Adam Miller"}};
    // ...
    employees[103] = {"Abigail Smith"};
    employees[104] = {"Jenna James"};
    // ...
    map<int,string>::iterator it = employees.begin();
    // ...
    while (it != employees.end())
    {
        // ...
        cout << it->first << ", " << it->second << endl;
        // ...
        it++;
    }
    return 0;
}
```

```
101, Jim Jones
102, Adam Miller
103, Abigail Smith
104, Jenna James
```

- Solution:

```
// Initialize map with initializer list (only at creation).
map<int,string> employees = {{101,"Jim Jones"},"{102,"Adam Miller"}};
// Add to map using operator [].
employees[103] = {"Abigail Smith"};
employees[104] = {"Jenna James"};
// Create iterator pointing to beginning of the map.
map<int,string>::iterator it = employees.begin();
// Display map contents using a while loop.
while (it != employees.end())
{
    // Print key-value pairs on one line each.
    cout << it->first << ", " << it->second << endl;
```



```
    // Move the iterator to the next map element.  
    it++;  
}
```

```
101, Jim Jones  
102, Adam Miller  
103, Abigail Smith  
104, Jenna James
```

9. Adding values to a map with insert

- `std::pair` can store two heterogeneous objects as a single unit. It's a special case of a `std::tuple` with two elements, with two member variables, `first` and `second`. `pair` is declared in `<utility>` (which is automatically included with `<map>`).
- When an element is stored in a map, it is stored as a `pair` object: the element's key is stored in the `first`, and the value is stored in the `second` member variable.
- The `map` class member function `insert` adds a `pair` object as an element to the map. You can use `make_pair(key, value)` to make a `pair`.
- In the following code, we insert three additional employee data in our existing non-empty map `emp`:

```
#include <iostream>  
#include <map>  
using namespace std;  
  
int main()  
{  
    // Create and initialize a map with a list.  
    map<int, string> emp = {{101, "Jim Jones"}, {105, "Adam Miller"}};  
  
    // Insert new key-value pairs in the map.  
    emp.insert(make_pair(102, "Beth Young"));  
    emp.insert(make_pair(103, "Jake Brown"));  
    emp.insert(make_pair(104, "John Smith"));  
    emp.insert(make_pair(105, "Peter Simon")); // duplicate key (ignored)  
    emp.insert(make_pair(106, "John Smith")); // duplicate value  
  
    // Iterate over the map and display the key-value pairs.  
    for(auto it=emp.begin(); it!=emp.end(); it++)  
    {  
        cout << it->first << ", " << it->second << endl;  
    }  
    return 0;  
}
```

```
101, Jim Jones  
102, Beth Young  
103, Jake Brown  
104, John Smith  
105, Adam Miller  
106, John Smith
```

10. Adding elements to a map with emplace

- Recall that `emplace` does not require objects to be created before manipulating them: instead, it constructs a new element in the container.
- You call the member function, pass the new element's key and value as arguments, and voilà:

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
    // Create and initialize a map with a list.
    map<int, string> emp = {{101, "Jim Jones"}, {105, "Adam Miller"}};

    // Insert new key-value pairs in the map.
    emp.emplace(102, "Beth Young");
    emp.emplace(103, "Jake Brown");
    emp.emplace(104, "John Smith");
    emp.emplace(105, "Peter Simon"); // duplicate key (ignored)
    emp.emplace(106, "John Smith"); // duplicate value

    // Iterate over the map and display the key-value pairs.
    for(auto it=emp.begin(); it!=emp.end(); it++)
    {
        cout << it->first << ", " << it->second << endl;
    }
    return 0;
}
```

```
101, Jim Jones
102, Beth Young
103, Jake Brown
104, John Smith
105, Adam Miller
106, John Smith
```

11. Finding values in a map with `find` using a key

- To find a value in a map, call the `find(key)` member function and pass a key. The function returns an iterator to the requested element or `end()`.
- Code example:

```
const map<int, string> students =
    {{1, "Alice"}, {2, "Bob"}, {3, "Charlie"}};

auto it = students.find(2); // returns iterator to key=2

if (it != students.end()) // check if it's in the map
    cout << it->second << endl;
else
    cout << "Key not found." << endl;
```

```
Bob
```

12. Retrieving values from a map with at using a key

- To retrieve a value from a map, call the `at(key)` member function and pass the key of the element associated with the desired value. If it exists, the value is returned, otherwise an exception is thrown.
- To prevent the exception to be thrown, call the `count(key)` function that returns only 1 (`true`) if the argument key exists.
- Code example: Using a map of employee IDs and names to find an employee.

```
map<int,string> emp =
    {{101,"Jim Jones"} , {102,"Adam Miller"},
     {103,"Beth Young"}, {104, "Jake Brown"}};

// Retrieve the employee with ID 103
if (emp.count(103))
    cout << emp.at(103) << endl;
else
    cout << "Employee not found.\n";

// Retrieve the employee with ID 109 (non-existing)
if (emp.count(109))
    cout << emp.at(109) << endl;
else
    cout << "Employee not found.\n";
```

```
Beth Young
Employee not found.
```

13. Difference between retrieval with find or at

- Both `map::find` and `map::at` access elements in a map by key but they differ in purpose, return type, and error behavior:
- Here is a detailed comparison:

Feature	<code>map::find(key)</code>	<code>map::at(key)</code>
Return type	iterator	reference to value
Missing key behavior	Returns <code>end()</code>	Throws <code>out_of_range</code>
Key existence check	Yes	No (use <code>count</code>)
Safe if key is missing	Yes	No
Exception-free	Yes	No
Modifies map?	No	No
Usable on <code>const</code> map	Yes	Yes
Typical use case	Conditional access	Guaranteed access

14. Deleting map elements

- You can delete an element with `erase(key)`. It returns 1 (true) if the key exists, or 0 (false) otherwise.
- Example:

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
    // Create and initialize a map with a list.
    map<int,string> emp =
        {{101,"Jim Jones"} , {102,"Adam Miller"},
         {103,"Beth Young"}, {104, "Jake Brown"}};

    // Delete the employee with the ID 103.
    emp.erase(103);

    // Display the content of the map.
    for (map<int,string>::iterator it = emp.begin();
         it != emp.end();
         it++)
        cout << it->first << ", " << it->second << endl;

    return 0;
}
```

```
101, Jim Jones
102, Adam Miller
104, Jake Brown
```

15. **PRACTICE** Using `emplace`, `find`, and `erase`

- Write a program using the starter code below to:
 1. Create an empty map `students` of student IDs and names.
 2. Using `emplace`, add three student IDs and their names: Alice (1001), Bob (1002), and Charlie (1004).
 3. Try to add a duplicate ID (1002) for James.
 4. Use `find` to check if a student with ID 1003 exists.
 5. If found, print their name.
 6. Remove the student with ID 1001 using `erase`.
 7. Print the contents of the map after deletion.
- Sample output:

```
Charlie
1002, James
1003, Charlie
```

- Starter code: tinyurl.com/student-map-emplace

```

// 1. Create an empty map `students` of student IDs and names
// 2. Using emplace, add three student IDs and their names:
//    Alice (1001), Bob (1002), and Charlie (1003)
// 3. Try to add a duplicate ID (1002) for James.
// 4. Use find to check if a student with ID 1003 exists.
// 5. If found, print their name.
// 6. Remove the student with ID 1001 using erase.
// 7. Print the contents of the map after deletion.

// Include the necessary header files.

// Use the std namespace

int main()
{
    // Create an empty map of student IDs and names.

    // Add three integer IDs and their names to the map.

    // Try to add a duplicate ID.

    // Use find to check if a student with ID 1003 exists.

    // If 1003 is found, print student name

    // Remove the student with ID 1001 using erase.

    // Print the contents of the map after deletion.

    return 0;
}

```

- Solution:

```

// Include the necessary header files.
#include <iostream>
#include <string>
#include <map>
// Use the std namespace
using namespace std;

int main()
{
    // Create an empty map of student IDs and names.
    map<int, string> students;

    // Add three integer IDs and their names to the map.
    students.emplace(1001, "Alice");
    students.emplace(1002, "Bob");
    students.emplace(1003, "Charlie");

    // Try to add a duplicate ID.
    students[1002] = {"James"};

    // Use find to check if a student with ID 1003 exists.
    auto it = students.find(1003);
    // If 1003 is found, print student name
    if (it != students.end())
        cout << it->second << endl;
    else
        cout << "ID not found.\n";
}

```

```
// Remove the student with ID 1001 using erase.
students.erase(1001);

// Print the contents of the map after deletion.
for (auto it=students.begin();it!=students.end();it++)
    cout << it->first << ", " << it->second << endl;

return 0;
}
```

```
Charlie
1002, James
1003, Charlie
```

16. Iterating over a map with a range-based loop

- If you don't want to mess with individual elements, you can use a range-based for loop to iterate over a map.
- The code below shows three different implementations of this loop:
 1. Using auto instead of the explicit data type.
 2. Using the class for a key-value pair, pair<keyType, valueType>.
 3. Using a structured binding declaration auto [key,value].

```
// Create a map of student IDs and their names
map<int,string> students =
    { {101, "Jill Johnson"}, {102, "James Henry"},
      {103, "Kayla Jones"}, {104, "Marie Jefferson"} };

// Display all student names with auto
for (auto s : students) cout << s.second << " ";
cout << endl;
// Display all student names with pair<keyType,valueType>
for (pair<int,string> s : students) cout << s.second << " ";
cout << endl;
// Display all student names with auto[k,v]
for (auto[k,v] : students) cout << v << " ";
```

```
Jill Johnson James Henry Kayla Jones Marie Jefferson
Jill Johnson James Henry Kayla Jones Marie Jefferson
Jill Johnson James Henry Kayla Jones Marie Jefferson
```

- The **structured binding declaration** defines two variables, k and v. Each time the loop iterates, a pair object from the students map is retrieved, the pair object is unpacked with the first member assigned to k and the second member assigned to v.

17. PRACTICE Structured bindings

- You're writing software for a vending machine. You have a map that links the ProductName (string) to its Price (double).
- Write a single range-based for loop using structured bindings ([name,price]) to display the menu.
- Sample output:

```

: --- Vending Machine Menu ---
: Product Chips      | Price: $1.75
: Product Pepsi     | Price: $1.25
: Product Snickers  | Price: $1.50
: Product Water     | Price: $1.00

```

- Starter code: tinyurl.com/vending-machine-menu

```

// Create a map of products and their prices
map<string, double> vendingMachine = {
    {"Snickers", 1.50},
    {"Pepsi", 1.25},
    {"Chips", 1.75},
    {"Water", 1.00}
};

cout << "--- Vending Machine Menu ---" << endl;

// TODO: Write a range-based for loop here using [product, price].
// It should print: "Product: [Name] | Price: $[Price]"

```

```

--- Vending Machine Menu ---

```

- Solution:

```

// Create a map of products and their prices
map<string, double> vendingMachine = {
    {"Snickers", 1.50},
    {"Pepsi", 1.25},
    {"Chips", 1.75},
    {"Water", 1.00}
};

cout << "--- Vending Machine Menu ---" << endl;

// TODO: Write a range-based for loop here using [product, price].
// It should print: "Product: [Name] | Price: $[Price]"
for (auto [product, price] : vendingMachine)
{
    cout << "Product "
        << left << setw(8) << product
        << " | Price: $"
        << fixed << setprecision(2) << price
        << endl;
}

```

```

--- Vending Machine Menu ---
Product Chips      | Price: $1.75
Product Pepsi     | Price: $1.25
Product Snickers  | Price: $1.50
Product Water     | Price: $1.00

```

18. Using an iterator with a map

- You have already seen examples using the member functions `begin()` and `end()` in an iterator.
- Here is an example of displaying the elements of a `map<string, string>` in reverse order using `crbegin()` and `crend()`, which implies a constant iterator starting at the end and moving backward through the map, in this case alphabetically.

```
// Create a map of capital cities for countries.
const map<std::string, std::string> capitals = {
    {"France", "Paris"},
    {"Germany", "Berlin"},
    {"Italy", "Rome"},
    {"Spain", "Madrid"},
    {"Canada", "Ottawa"},
    {"Japan", "Tokyo"},
    {"Australia", "Canberra"},
    {"Brazil", "Brasilia"},
    {"India", "New Delhi"},
    {"Kenya", "Nairobi"}
};

// Create an iterator.
map<string, string>::const_reverse_iterator it;

// Use the iterator to display each element in the map in reverse.
for(it = capitals.crbegin(); it != capitals.crend(); it++)
{
    cout << setw(10)
        << it->second << " is the capital of " << it->first << endl;
}
```

```
Madrid is the capital of Spain
Nairobi is the capital of Kenya
Tokyo is the capital of Japan
Rome is the capital of Italy
New Delhi is the capital of India
Berlin is the capital of Germany
Paris is the capital of France
Ottawa is the capital of Canada
Brasilia is the capital of Brazil
Canberra is the capital of Australia
```

- When you declare a map like `capitals`, there is actually a hidden third parameter called the **comparator**. Behind the scenes, the computer sees this:

```
std::map<std::string, std::string, std::less<std::string>> capitals;
```

- The default comparator `std::less` ensures that for any two keys `a, b` if `a < b`, `a` comes before `b`. For `std::string`, `<` is an alphabetical comparison.

19. PRACTICE Using an iterator with a map

- Modify the starter code so that you get this output:

```
Canberra is the capital of Australia
Brasilia is the capital of Brazil
Ottawa is the capital of Canada
```



```
Paris is the capital of France
Berlin is the capital of Germany
New Delhi is the capital of India
Rome is the capital of Italy
Tokyo is the capital of Japan
Nairobi is the capital of Kenya
Madrid is the capital of Spain
```

- Starter code: <https://tinyurl.com/capital-cities>
- Solution:

```
// Create a map.  const map<std::string, std::string> capitals = {
{"France", "Paris"}, {"Germany", "Berlin"}, {"Italy", "Rome"},
{"Spain", "Madrid"}, {"Canada", "Ottawa"}, {"Japan", "Tokyo"},
{"Australia", "Canberra"}, {"Brazil", "Brasilia"}, {"India", "New
Delhi"}, {"Kenya", "Nairobi"} };

// Create an iterator.  map<string,string>::const_iterator it;

// Use the iterator to display each element in the map
for(it = capitals.begin(); it != capitals.end(); it++)
{
    cout << setw(10) <<
        it->second << " is the capital of " << it->first << endl;
}
```

```
Canberra is the capital of Australia
Brasilia is the capital of Brazil
Ottawa is the capital of Canada
Paris is the capital of France
Berlin is the capital of Germany
New Delhi is the capital of India
Rome is the capital of Italy
Tokyo is the capital of Japan
Nairobi is the capital of Kenya
Madrid is the capital of Spain
```

20. Storing a vector as values in a map

- In this map, the keys are string student names, and the values are vector objects containing their test scores.
- To display the scores, we use a range-based for loop, and nested within, another for loop that uses the operator [].

```
#include <iostream>
#include <string>
#include <vector>
#include <map>
using namespace std;

int main()
{
    // Create vectors to hold test scores.
    vector<int> scores1 = {88, 92, 100};
    vector<int> scores2 = {95, 74, 81};
```

```

vector<int> scores3 = {72, 88, 91};
vector<int> scores4 = {70, 75, 78};

// Create a map to hold all the test scores.
map<string, vector<int>> testScores;
testScores["Kayla"] = scores1;
testScores["Luis"] = scores2;
testScores["Sophie"] = scores3;
testScores["Ethan"] = scores4;

// Display each student's test scores.
for (auto element : testScores)
{
    // Display the student name.
    cout << "Student: " << element.first;

    // Display the student's test scores.
    for (int i=0; i < element.second.size(); i++)
    {
        cout << "\t" << element.second[i] << " ";
    }
    cout << endl;
}
return 0;
}

```

Student: Ethan	70	75	78
Student: Kayla	88	92	100
Student: Luis	95	74	81
Student: Sophie	72	88	91

- The loop over the vector elements can be simplified because we're not using the index information:

```

#include <iostream>
#include <string>
#include <vector>
#include <map>
using namespace std;

int main()
{
    // Create vectors to hold test scores.
    vector<int> scores1 = {88, 92, 100};
    vector<int> scores2 = {95, 74, 81};
    vector<int> scores3 = {72, 88, 91};
    vector<int> scores4 = {70, 75, 78};

    // Create a map to hold all the test scores.
    map<string, vector<int>> testScores;
    testScores["Kayla"] = scores1;
    testScores["Luis"] = scores2;
    testScores["Sophie"] = scores3;
    testScores["Ethan"] = scores4;

    // Display each student's test scores.
    for (auto element : testScores)
    {
        // Display the student name.
        cout << "Student: " << element.first;

        // Display the student's test scores.
    }
}

```

```

        for (auto score : element.second)
        {
            cout << "\t" << score << " ";
        }
        cout << endl;
    }
    return 0;
}

```

Student: Ethan	70	75	78
Student: Kayla	88	92	100
Student: Luis	95	74	81
Student: Sophie	72	88	91

21. **PRACTICE** Student scores with structured binding

- We can do even better!
- Rewrite the last program so that you use structured binding for the outer loop over testScores. Starter code: tinyurl.com/structured-binding
- Solution:

```

#include <iostream>
#include <string>
#include <vector>
#include <map>
using namespace std;

int main()
{
    // Create vectors to hold test scores.
    vector<int> scores1 = {88, 92, 100};
    vector<int> scores2 = {95, 74, 81};
    vector<int> scores3 = {72, 88, 91};
    vector<int> scores4 = {70, 75, 78};

    // Create a map to hold all the test scores.
    map<string, vector<int>> testScores;
    testScores["Kayla"] = scores1;
    testScores["Luis"] = scores2;
    testScores["Sophie"] = scores3;
    testScores["Ethan"] = scores4;

    // Display each student's test scores.
    for (const auto& [name,scores] : testScores)
    {
        // Display the student name.
        cout << "Student: " << name;

        // Display the student's test scores.
        for (int score : scores)
        {
            cout << "\t" << score << " ";
        }
        cout << endl;
    }
    return 0;
}

```

Student: Ethan	70	75	78
Student: Kayla	88	92	100
Student: Luis	95	74	81
Student: Sophie	72	88	91

- Why this is better:
 1. Readability: You get to name your variables name and scores immediately instead of dealing with the generic `element.first` and `element.second`.
 2. Cleanliness: It looks similar to Python's dictionary unpacking.
 3. Using `auto&` to stop C++ from making a copy of the vector scores for every iteration. Copying vectors is slow.
 4. Using `const auto&` because we're not modifying the data only printing it. `const` stops us from making accidental changes.

22. **PRACTICE** Exercise: Inventory Prices

- Write a program that:
 1. Creates a `map<string, vector<double>>` to store item prices from multiple suppliers.
 2. Each key is a **product name** (e.g. "Wrench", "Hammer", "Drill").
 3. Each value is a vector of **prices** for that product from different suppliers.
- Use a range-based for loop to:
 - Print each **product name**
 - Print each **supplier price** on the same line using a nested for loop

- Sample output:

```
Item: Drill      Prices:  45.00 42.99 47.50
Item: Hammer    Prices:   8.50  9.10  8.75
Item: Wrench     Prices:  12.99 11.49 13.25
```

- Requirements:

- Use brace `{}` initialization for vectors.
- Use `auto` in your loops.
- Print double values with two decimal places.
- **Bonus:** After listing each item's prices, also display the **lowest** price for that item. Compute that value using a function. Output:

```
: Item: Drill    Prices:  45.00 42.99 47.50  Lowest: 42.99
: Item: Hammer  Prices:   8.50  9.10  8.75  Lowest:  8.50
: Item: Wrench   Prices:  12.99 11.49 13.25  Lowest: 11.49
```

- **Questions:** What does `auto` stand for in each of the two (nested) loops of this program?
- **Solution:** We'll discuss it in class once you've tried it yourself.

23. Inventory Prices: Solutions

- Without bonus:

```

// Create a map to store item prices for different suppliers.
// Header files
#include <iostream> // want to print stuff
#include <string> // item names are stored as strings
#include <map> // we store items in a map of prices
#include <vector> // we store prices as double vector values
#include <iomanip> // we want to manipulate I/O formatting

int main() {
    // Create map<string,vector<double>> to store item prices from
    // multiple suppliers.
    std::map<std::string,std::vector<double>> inventory = {
        // Each key is a product name (e.g. "Wrench", "Hammer", "Drill").
        // Each value is a vector of prices for that product from
        // different suppliers.
        {"Drill", {45.00, 42.99, 47.50}},
        {"Hammer", {8.50, 9.10, 8.75}},
        {"Wrench", {12.99, 11.49, 13.25}}
    };

    // Use a range-based for loop to: Print each product name, and print
    // each supplier price on the same line using a nested for loop
    // Q: what is `auto`?
    for (const auto& item : inventory)
    {
        // Display the item (one fixed-width column for "Item: <name>"
        // Arrange text left aligned (this is a little tricky)
        std::cout << std::left << std::setw(13)
            << ("Item: " + item.first) << "Prices: ";
        // Display the prices from different suppliers
        // Q: what is `auto`?
        for (const auto& price : item.second)
        {
            // Format: fixed length, two numbers after decimal point
            std::cout << std::fixed << std::setprecision(2)
                << std::right << std::setw(5)
                << price << " ";
        }
        // Next item
        std::cout << std::endl;
    }
    return 0;
}

```

```

Item: Drill  Prices: 45.00 42.99 47.50
Item: Hammer Prices:  8.50  9.10  8.75
Item: Wrench Prices: 12.99 11.49 13.25

```

- With bonus: hand-crafted function

```

// Create a map to store item prices for different suppliers.
// Header files
#include <iostream> // want to print stuff
#include <string> // item names are stored as strings
#include <map> // we store items in a map of prices
#include <vector> // we store prices as double vector values
#include <iomanip> // we want to manipulate I/O formatting

// function declaration
double getLowestPrice(const std::vector<double>&);

```

```

int main() {
    // Create map<string,vector<double>> to store item prices from
    // multiple suppliers.
    std::map<std::string,std::vector<double>> inventory = {
        // Each key is a product name (e.g. "Wrench", "Hammer", "Drill").
        // Each value is a vector of prices for that product from
        // different suppliers.
        {"Drill", {45.00, 42.99, 47.50}},
        {"Hammer", {8.50, 9.10, 8.75}},
        {"Wrench", {12.99, 11.49, 13.25}}
    };

    // Use a range-based for loop to: Print each product name, and print
    // each supplier price on the same line using a nested for loop
    // Q: what is `auto`?
    for (const auto& item : inventory)
    {
        // Display the item (one fixed-width column for "Item: <name>")
        // Arrange text left aligned (this is a little tricky)
        std::cout << std::left << std::setw(13)
            << ("Item: " + item.first) << "Prices: ";
        // Display the prices from different suppliers
        // Q: what is `auto`?
        for (const auto& price : item.second)
        {
            // Format: fixed length, two numbers after decimal point
            std::cout << std::fixed << std::setprecision(2)
                << std::right << std::setw(5)
                << price << " ";
        }
        // Next item
        std::cout << "Lowest: " << std::right << std::setw(5)
            << getLowestPrice(item.second);
        std::cout << std::endl;
    }
    return 0;
}
// function definition
double getLowestPrice(const std::vector<double>& prices)
{
    double lowest = prices[0];
    for(int i=1;i<prices.size();i++)
        if(prices[i]<lowest) lowest=prices[i];
    return lowest;
}

```

```

Item: Drill  Prices: 45.00 42.99 47.50 Lowest: 42.99
Item: Hammer Prices:  8.50  9.10  8.75 Lowest:  8.50
Item: Wrench Prices: 12.99 11.49 13.25 Lowest: 11.49

```

- With bonus: <algorithm> function

```

// Create a map to store item prices for different suppliers.
// Header files
#include <iostream> // want to print stuff
#include <string> // item names are stored as strings
#include <map> // we store items in a map of prices
#include <vector> // we store prices as double vector values
#include <iomanip> // we want to manipulate I/O formatting
#include <algorithm> // for the min_element function

```

```

// function declaration
double getLowestPrice(const std::vector<double>&);

int main() {
    // Create map<string,vector<double>> to store item prices from
    // multiple suppliers.
    std::map<std::string,std::vector<double>> inventory = {
        // Each key is a product name (e.g. "Wrench", "Hammer", "Drill").
        // Each value is a vector of prices for that product from
        // different suppliers.
        {"Drill", {45.00, 42.99, 47.50}},
        {"Hammer", {8.50, 9.10, 8.75}},
        {"Wrench", {12.99, 11.49, 13.25}}
    };

    // Use a range-based for loop to: Print each product name, and print
    // each supplier price on the same line using a nested for loop
    // Q: what is `auto`?
    for (const auto& item : inventory)
    {
        // Display the item (one fixed-width column for "Item: <name>"
        // Arrange text left aligned (this is a little tricky)
        std::cout << std::left << std::setw(13)
            << ("Item: " + item.first) << "Prices: ";
        // Display the prices from different suppliers
        // Q: what is `auto`?
        for (const auto& price : item.second)
        {
            // Format: fixed length, two numbers after decimal point
            std::cout << std::fixed << std::setprecision(2)
                << std::right << std::setw(5)
                << price << " ";
        }
        // Next item
        std::cout << "Lowest: " << std::right << std::setw(5)
            << getLowestPrice(item.second);
        std::cout << std::endl;
    }
    return 0;
}

// function definition
double getLowestPrice(const std::vector<double>& prices)
{
    if (prices.empty()) return 0.0; // handle empty vector case
    // iterator contains the minimum computation
    auto min_it = std::min_element(prices.begin(), prices.end());
    return *min_it;
}

```

```

Item: Drill  Prices: 45.00 42.99 47.50 Lowest: 42.99
Item: Hammer Prices:  8.50  9.10  8.75 Lowest:  8.50
Item: Wrench Prices: 12.99 11.49 13.25 Lowest: 11.49

```

24. Storing objects of a class as values in a map

- In addition to storing string values, primitive values, and vector objects, you can also store class objects as long as the class has a **default constructor**.
- Example: The Contact class holds a person's name and email address. We're storing it in a file `Contact.h`, which we're going to use in a `map<string, Contacts>` with the Contact object's name as key field.

- The Contact class in Contact.h:

```
#ifndef CONTACT_H
#define CONTACT_H
#include <string>
using namespace std;

class Contact
{
private:
    string name;
    string email;
public:
    Contact() {name=""; email="";}
    Contact(string n, string em) {name=n;email=em;}
    void setName(string n){name=n;}
    void setEmail(string em){email=em;}
    string getName() const {return name;}
    string getEmail() const {return email;}
};
#endif
```

- In the main program below, we create three Contact objects, and a map contacts to hold them with the object's name as key. Then we use find to search for a name that we get from the commandline.
- **Remember:** The function getline(cin, searchName) gets input (including whitespace) from the command-line and stores it in searchName.
- Open your editor now on tinyurl.com/contacts-map and code along with me!

```
#include <iostream>
#include <string>
#include <map>
#include "Contact.h"
using namespace std;

int main()
{
    string searchName; // The name to search for

    // Create some Contact objects
    Contact contact1("Ashley Miller", "amiller@faber.edu");
    Contact contact2("Jacob Brown", "jbrown@gotham.edu");
    Contact contact3("Emily Ramirez", "eramirez@coolidge.edu");

    // Create a map to hold the Contact objects.
    map<string, Contact> contacts;

    // Add the Contact objects to the map (key = name)
    contacts[contact1.getName()] = contact1;
    contacts[contact2.getName()] = contact2;
    contacts[contact3.getName()] = contact3;

    // Get the name to search for.
    cout << "Enter a name to search for: ";
    getline(cin, searchName);
    cout << searchName << endl;

    // Search for the name in the map.
    auto iter = contacts.find(searchName);

    // Display the results.
```



```

if (iter != contacts.end())
{
    cout << "Name: " << iter->second.getName() << endl
        << "Email: " << iter->second.getEmail() << endl;
}
else
{
    cout << "Contact not found.\n";
}
return 0;
}

```

- Input: searchName

```

cd ../data
echo "Billy Clark" > searchName
cat searchName

```

Billy Clark

25. **PRACTICE** Exercise: Storing Class Objects in a Map

- Write a program that:
 1. Defines a Book class with the following private members:
 - title (string)
 - author (string)
 - year (int)
 2. Include a default constructor and a parameterized constructor.
 3. Include public getters for all members.
 4. In main():
 - Create 3 Book objects
 - Store them in a map<string, Book> using the book title as the key
 5. Ask the user to enter a book title.
 - Use getline(cin, title) to allow spaces
 - Search the map using find
 - If found, print author and year; otherwise, print “Book not found.”

- **Example output:**

```

Enter a book title to search for: The Great Gatsby
Author: F. Scott Fitzgerald
Year: 1925

```

- Requirements:

- Class definition in a separate header file (Book.h)
- Use map<string, Book>
- Use find and a range-based for loop only if extending
- **Solution:** My sample solution has 20 lines of code only. We'll discuss this in class after you've given it a try.

26. Solution: Storing Book class objects in a map

- The Book class is defined in a header file Book.h:

```
#ifndef BOOK_H
#define BOOK_H

#include <string>

class Book {
private:
    std::string title;
    std::string author;
    int year;

public:
    // Default Constructor
    Book();

    // Parameterized Constructor
    Book(std::string t, std::string a, int y);

    // Getters
    std::string getTitle() const;
    std::string getAuthor() const;
    int getYear() const;
};

#endif
```

- The Book class member functions are defined in a C++ file Book.cpp, which includes Book.h:

```
#include "Book.h"

// Default Constructor
Book::Book() {
    title = "";
    author = "";
    year = 0;
}

// Parameterized Constructor
Book::Book(std::string t, std::string a, int y) {
    title = t;
    author = a;
    year = y;
}

// Getters
std::string Book::getTitle() const {
    return title;
}

std::string Book::getAuthor() const {
    return author;
}

int Book::getYear() const {
    return year;
}
```

- The main program BookTest.cpp handles the std::map logic, user input, and searching:

```

#include <iostream>
#include <string>
#include <map>
#include "Book.cpp"

int main() {
    // Create 3 Book objects
    Book b1("The Great Gatsby", "F. Scott Fitzgerald", 1925);
    Book b2("1984", "George Orwell", 1949);
    Book b3("The Hobbit", "J.R.R. Tolkien", 1937);

    // Store them in a map using the title as the key
    std::map<std::string, Book> library;

    // We insert pairs into the map.
    // Key = Book Title, Value = Book Object
    library[b1.getTitle()] = b1;
    library[b2.getTitle()] = b2;
    library[b3.getTitle()] = b3;

    // Ask user for input
    std::string searchTitle;
    std::cout << "Enter a book title to search for: ";

    // getline is used to ensure we capture titles with spaces
    std::getline(std::cin, searchTitle);
    std::cout << searchTitle << std::endl;

    // Search the map using find 'it' is an iterator. If found, it
    // points to the pair. If not found, it points to library.end()
    std::map<std::string, Book>::iterator it = library.find(searchTitle);

    if (it != library.end()) {
        // Found the book
        // it->second accesses the Value (the Book object)
        Book foundBook = it->second;

        std::cout << "Author: " << foundBook.getAuthor() << std::endl;
        std::cout << "Year: " << foundBook.getYear() << std::endl;
    } else {
        // Not found
        std::cout << "Book not found." << std::endl;
    }
    return 0;
}

```

- Inputfile searchBook:

```

cd ../data
echo "The Great Gatsby" > searchBook
cat searchBook

```

```
The Great Gatsby
```

27. Storing objects of a class as a key to a map

- You can use class objects as keys in a map as long as the class has overloaded the < operator to allow comparing objects with it.

- Example: A Customer class. The < operator is overloaded: When you compare two Customer objects with this operator, then the custNumber is compared and this comparison counts for one customer being "below" another. Note that other logical operators are not defined for Customer.

```
#ifndef CUSTOMER_H
#define CUSTOMER_H
#include <iostream>
using namespace std;

class Customer {
private:
    int custNumber;
    string name;
public:
    Customer(int cn, string n) : custNumber(cn), name(n){}
    void setCustNumber(int cn) {custNumber = cn;}
    void setName(string& n) {name=n;}
    int getCustNumber() const {return custNumber;}
    string getName() const {return name;}

    bool operator < (const Customer &right) const
    { bool status = false;
      if (custNumber < right.custNumber)
          status = true;
      return status; }
};
#endif
```

- Testing the class:

```
#include <iostream>
#include "Customer.h"
using namespace std;

int main()
{
    Customer Paul(1001, "Paul Bettany");
    Customer Jenny(1002, "Jennifer Connelly");
    // Check the operator < overloading
    cout << boolalpha << (Paul < Jenny) << endl;
    cout << boolalpha << (Jenny < Paul) << endl;

    return 0;
}
```

- Example: We can now write a program that assigns seats in a theatre to customers and use Customer objects as the keys in a map. The values are the seats (as strings).

```
#include <iostream>
#include <map>
#include <iomanip>
#include "Customer.h"
using namespace std;

int main()
{
    // Create three Customer objects.
    Customer customer1(1001, "Frodo Baggins");
```

```

Customer customer2(1002, "Sam Gamgee");
Customer customer3(1003, "Pippin Tuk");

// Create a map to hold the seat assignments.
map<Customer, string> assignments;

// Use the map to store the seat assignments.
assignments[customer1] = "1A";
assignments[customer2] = "2B";
assignments[customer3] = "3C";

// Define a map iterator.
map<Customer, string>::iterator it;

// Display all objects in the map.
for (auto it = assignments.begin();
     it != assignments.end(); it++)
{
    cout << "Customer No. "
          << it->first.getCustNumber()
          << ", " << setw(5)
          << "Name: " << it->first.getName()
          << ", " << "\tSeat: "
          << it->second << endl;
}
return 0;
}

```

28. **PRACTICE** Storing class information as map key

- The last program can be simplified: instead of creating Customer objects and then assigning them to an empty map, we can create a map to hold the seat assignments right away using an initializer list.
- Use the starter code to simplify the program by initializing the map on one line. Change the output to use auto and a range-based loop so that the output looks like this:
- Starter code: onecompiler.com/cpp/43rk3vam4

```

// Create an assignment map that uses Customer data as key
// to store the customer's seating assignment.
#include <iostream>
#include <map>
#include <iomanip>
using namespace std;

// Customer class definition
class Customer {
private:
    int custNumber;
    string name;
public:
    Customer(int cn, string n) : custNumber(cn), name(n){}
    void setCustNumber(int cn) {custNumber = cn;}
    void setName(string& n) {name=n;}
    int getCustNumber() const {return custNumber;}
    string getName() const {return name;}
    bool operator < (const Customer &right) const
    { bool status = false;
      if (custNumber < right.custNumber)
        status = true;
      return status; }
};

```

```

int main()
{
    // Create three Customer objects.
    Customer customer1(1001, "Frodo Baggins");
    Customer customer2(1002, "Sam Gamgee");
    Customer customer3(1003, "Pippin Tuk");

    // Create a map to hold the seat assignments.
    map<Customer, string> assignments;

    // Use the map to store the seat assignments.
    assignments[customer1] = "1A";
    assignments[customer2] = "2B";
    assignments[customer3] = "3C";

    // Define a map iterator.
    map<Customer, string>::iterator it;

    // Display all objects in the map.
    for (auto it = assignments.begin();
         it != assignments.end(); it++)
    {
        cout << "Customer No. "
              << it->first.getCustNumber()
              << ", " << setw(5)
              << "Name: " << it->first.getName()
              << ", " << "\tSeat: "
              << it->second << endl;
    }
    return 0;
}

```

```

Customer No. 1001, Name: Frodo Baggins,      Seat: 1A
Customer No. 1002, Name: Sam Gamgee,        Seat: 2B
Customer No. 1003, Name: Pippin Tuk,        Seat: 3C

```

- Solution: onecompiler.com/cpp/43rk4hnmk

```

// Create an assignment map that uses Customer data as key
// to store the customer's seating assignment.
#include <iostream>
#include <map>
#include <iomanip>
using namespace std;

// Customer class definition
class Customer {
private:
    int custNumber;
    string name;
public:
    Customer(int cn, string n) : custNumber(cn), name(n){}
    void setCustNumber(int cn) {custNumber = cn;}
    void setName(string& n) {name=n;}
    int getCustNumber() const {return custNumber;}
    string getName() const {return name;}
    bool operator < (const Customer &right) const
    { bool status = false;
      if (custNumber < right.custNumber)
        status = true;
    }
}

```

```

        return status; }
};

int main()
{
    // Create a map to hold the seat assignments
    map<Customer, string> assignments =
    { {Customer(1001, "Frodo Baggins"), "1A"},
      {Customer(1002, "Samwise Gamgee"), "2B"},
      {Customer(1003, "Pippin Tuk"), "3C"} };

    // Display all objects in the map.
    for (auto element : assignments)
    {
        cout << element.first.getName() << "\t"
              << element.second << endl;
    }
    return 0;
}

```

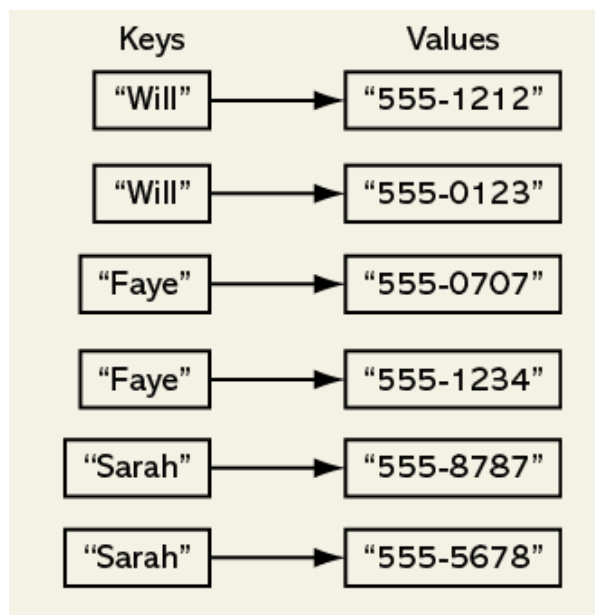
```

Frodo Baggins    1A
Samwise Gamgee   2B
Pippin Tuk       3C

```

29. Other map classes: unordered_map, multimap, set

- The unordered_map is the basis of the hash table. Its keys are not sorted and it has better performance for large maps. Otherwise working with it is just like map except that you include <unordered_map> and use the unordered_map keyword. This data structure is the basis of hash tables (vs. trees).
- The multimap class allows duplicate (ordered) keys. It uses the <map> directive. For example in a phone book application, you could store names and phone numbers as values. If a person has multiple phone numbers, you can store multiple elements using their name as key.



- A limitation of the multimap is that the `[]` operator is not overloaded so that you cannot add elements using it. Use `emplace` or `insert` instead.
- The multimap class uses the **binary tree** data structure, while the `unordered_multimap` uses a **hash table**.

30. Review questions

1. Each element that is stored in a map has two parts. What are they?

Each map element has a key and a value.

2. Write a statement that defines a map named `myMap`. The keys in `myMap` should be integers, and the values should be strings. Code an example that works.

```
map<int,string> myMap;
```

3. Suppose an empty map named `employee` has been created. What does the following statement do? Code a complete example.

```
employee[543] = "Joanne Manchester";
```

The statement adds an element `{543, "Joanne Manchester"}` to the map. Its key is 543 and the string is the value of the element.

4. Describe two ways in which you can retrieve an element with a particular key `k` in a map called `myMap`. Code an example.

- You can call `myMap.at(k)`, which will return the value for the key.
- You can call `myMap.find(k)`, which returns an iterator to the element.

5. If you want to store objects of a class you have written, as values in a map, what do you have to make sure that the class has?

To store your own class objects in a map, the class must have a default constructor.

6. What is the difference between a map and an `unordered_map`?

The keys of the `unordered_map` are not sorted, and the data structure performs better.

7. What is the difference between a map and a multimap?

The multimap allows duplicate keys.

31. Glossary

TERM	EXPLANATION
Map	Associative container of key-value pairs
<code>iterator->first</code>	Access the key element of map element
<code>iterator->second</code>	Access the value element of a map element

TERM	EXPLANATION
<code>map<T,T> name</code>	Default constructor for a map
<code>map<T,T> name(other_map)</code>	Copy constructor for map
<code>map<T,T> name(it1,it2)</code>	Range constructor for map
<code>emplace(key, value)</code>	Inserts an element constructed in-place
<code>insert(make_pair(k,v))</code>	Inserts an existing pair object
<code>at(key)</code>	Accesses value by key; throws if key not found
<code>find(key)</code>	Returns iterator to key or <code>end()</code> if not found
<code>count(key)</code>	Returns 1 if key exists, 0 otherwise
<code>erase(key)</code>	Removes element by key; returns 1 if found
<code>crbegin(), crend()</code>	Return const reverse iterators for map traversal
<code>auto [k, v]</code>	Structured binding for unpacking map pairs
<code>[] (operator)</code>	Accesses or inserts (or creates) value by key
Class as map key	Custom objects can be keys if <code><</code> is overloaded
<code>operator<</code>	Required to use custom class as map key
<code>map<Class, T></code>	Declares a map with class objects as keys
initializer list for map	Inline using brace-enclosed {key, value} pairs
Range-based loop over map	<code>for (auto [k,v] : map) unpacks key/value</code>

Author: Marcus Birkenkrahe

Created: 2026-04-25 Sat 08:21